

ORIGIN AI: A Tiered Multi-Subject Context Engine for Low-Cost Pedagogical Mentorship

A report submitted for the course named Project - I (CS3201)

Submitted By

S NAVEEN
6TH SEMESTER
230101099

Supervised By

DR. NONGMEIKAPAM KISHORJIT SINGH



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR
APRIL, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

S Naveen
230101099

DATE:



Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Nongmeikapam Kishorjit Singh
Assistant Professor, CSE

Email: kishorjit@iiitmanipur.ac.in
Contact No: +91 8974008610

To Whom It May Concern

This is to certify that the project report entitled "**ORIGIN AI: A Tiered Multi-Subject Context Engine for Low-Cost Pedagogical Mentorship**", submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is record bonafide work carried out by **S Naveen** bearing roll number 230101099.

Signature of Supervisor

(Dr. Nongmeikapam Kishorjit Singh)

Signature of the Examiner 1:

Signature of the Examiner 2:

Signature of the Examiner 3:

Signature of the Examiner 4:

Abstract

With the proliferation of Large Language Models (LLMs) in educational technology, the challenges of operational cost, latency, and contextual relevance have become critical barriers to scaling personalized mentorship. This project introduces **Origin AI**, a context-aware, multimodal AI mentor designed for high-stakes competitive examination preparation. Origin AI implements a novel **4-Layer Context Assembly** model that integrates student history, real-time UI state, visual highlights, and multimodal queries to provide in-situ assistance without manual context description. To address financial sustainability, we propose a **Tiered Orchestration Pipeline** that prioritizes zero-cost local assets (stored solutions and subject-specific knowledge bases) before escalating to synthetic LLM generation. Experimental results demonstrate an 85% reduction in token consumption and a 73% reduction in median latency, while achieving 96% subject resolution accuracy via a scoring-based **Cross-Subject Gate**. The system also incorporates pedagogical guardrails to enforce academic integrity and hint-first mentorship, ensuring that the AI functions as a teacher rather than a solution-providing proxy.

Acknowledgement

I would like to express my deepest gratitude to my supervisor, **Dr. Nongmeikapam Kishorjit Singh**, for his invaluable guidance, technical insights, and constant encouragement throughout the development of this project. His expertise in AI and pedagogy was instrumental in refining the tiered orchestration logic and the context assembly model. I also thank the Department of Computer Science and Engineering at IIIT Manipur for providing the resources and environment necessary to conduct this research. Finally, I am grateful to my family and friends for their unwavering support during the countless hours spent on implementation and documentation.

S Naveen
230101099

Professor's Word on the Project



Figure 1: Dr. Nongmeikapam Kishorjit Singh, Assistant Professor, CSE, IIIT Manipur

“Genuine and most appropriate AI assisted teaching-learning for students preparing the competitive exams. Also it gives voice assisted suggestions and instructions in English and Hindi. It's Truly Worthy AI tool and can be explored by the aspirants.”

Contents

List of Figures	6
List of Tables	7
1 Introduction	8
1.1 Motivation	8
1.2 Introduction to AI in Education	8
1.3 The Crisis of AI Scaling in EdTech	9
1.4 Problem Statement – Three Core Deficits	9
1.5 Proposed Solution: Origin AI	10
1.6 Project Subgoals (Detailed Version-by-Version History)	10
1.7 Development Methodology: Cyclic Waterfall Model	11
1.8 Project Timeline (Gantt Chart)	13
1.9 Comparative Analysis of Accessing Platforms	13
1.10 Current Trends in AI-Driven Education	13
1.11 Previous Projects and Similar Platforms	14
1.12 Challenges and Opportunities	14
2 Literature Survey	15
2.1 Theoretical Framework: The TPACK Model	15
2.2 Traditional RAG Systems and Their Limitations	16
2.3 Rule-Based Pedagogical Agents	16
2.4 Generic LLM Chatbots in Education	16
2.5 Context-Aware Systems and In-Situ Assistance	17
2.6 The Rise of Multimodal Learning Environments	17
2.7 Speech-to-Text (STT) and Text-to-Speech (TTS) in Pedagogy	17
2.8 Vector Databases and Semantic Memory	18
2.9 Gaps in Current Research – Our Contribution	18
3 Requirement Engineering	19
3.1 Functional Requirements	19
3.2 Non-Functional Requirements	19
3.3 Scope and Non-Goals	20
3.3.1 In Scope	20
3.3.2 Out of Scope	20
3.4 User Personas	20
3.5 Detailed Use Cases	21
3.5.1 Use Case 1: In-Situ Doubt Resolution	21
3.5.2 Use Case 2: Academic Integrity Enforcement	21
3.6 Hardware and Software Requirements	21
3.6.1 Development Environment	21
3.6.2 Production Environment (Deployment)	21

4	System Design	22
4.1	System Architecture Overview	22
4.2	The 4-Layer Context Assembly Model	23
4.2.1	Layer 1: GlobalContext (Long-Term Memory)	23
4.2.2	Layer 2: PageContext (Live Environmental State)	23
4.2.3	Layer 3: HighlightContext (Granular Selection)	23
4.2.4	Layer 4: UserQuery (Active Intent)	23
4.3	Tiered Orchestration Pipeline	24
4.4	Mathematical Modeling of Cost and Relevance	24
4.5	Cross-Subject Gating Logic	24
4.6	Pedagogical Guardrails and Policy Resolution	24
4.7	Request Processing Sequence	25
4.8	Database Schema Design	25
4.8.1	Conceptual Data Model	25
4.8.2	Physical Database Schema	27
4.9	System Data Flow Analysis	28
4.9.1	Level 0 Context Diagram	28
4.9.2	Level 1 Process Decomposition	29
4.10	Role-Based Access Flow	30
5	Implementation	31
5.1	Backend Implementation with FastAPI	31
5.1.1	Microservice Architecture	31
5.1.2	PostgreSQL with pgvector	31
5.2	Frontend Engineering with Next.js	31
5.2.1	Floating Mentor UI	31
5.2.2	Highlight Capture Engine	32
5.2.3	Multimodal Integration	32
5.3	Deployment and Infrastructure	32
5.3.1	Containerization	32
5.3.2	Cloud Hosting	32
5.4	Security and API Protection	32
5.4.1	Authentication Tunneling	32
5.4.2	Rate Limiting	32
5.5	Technical Deep Dive: Context Assembly	33
5.6	Implementation of the Tiered Pipeline	33
5.7	Unified Math Rendering Logic	33
6	Results	35
6.1	Experimental Setup and Test Dataset	35
6.2	Cost-Efficiency Benchmarks	35
6.3	Latency Analysis	35
6.4	Subject Resolution Accuracy	36
6.5	User Satisfaction Study	36
6.6	Comparative Performance Analysis	36
6.7	Error Analysis and Edge Cases	36
6.8	Ablation Study	36
6.9	Scalability and Stress Testing	37
6.10	Conclusion of Results	37
6.11	Real-World Use Cases	37
6.11.1	Case Study: Midnight Doubt Resolution	37
6.11.2	Case Study: Mock Test Proctoring	37

7	Security and Privacy	38
7.1	Security Philosophy	38
7.2	Authentication Tunneling	38
7.3	Data Privacy and PII Protection	38
7.4	Rate-Limiting and Anti-Abuse	38
8	Infrastructure and Deployment	40
8.1	Infrastructure Overview	40
8.2	Containerization Strategy	40
8.3	Docker Compose Orchestration	40
8.4	Cloud Hosting and Scaling	40
9	Conclusion	42
9.1	Summary of Technical Contributions	42
9.2	Social Impact and Accessibility	42
9.3	Limitations of Current Implementation	42
9.4	Future Work	43
	Peer Reviews	44
	Bibliography	45

List of Figures

1	Dr. Nongmeikapam Kishorjit Singh, Assistant Professor, CSE, IIIT Manipur	2
1.1	Cyclic Waterfall Model Architecture for Origin AI	12
1.2	Development Timeline for Origin AI Phases (Days 01-24)	13
2.1	The TPACK Framework (Source: [2])	15
4.1	Origin AI Technical Architecture	22
4.2	Database Schema Conceptual Model (Object-Oriented Notation)	26
4.3	Physical Database Implementation Schema	27
4.4	Data Flow Diagram - Level 0 (Context Diagram)	28
4.5	Data Flow Diagram - Level 1 (Process Decomposition)	29
4.6	Role-Based Access and Data Flow	30

List of Tables

1.1	Comparative Analysis of AI Mentorship Solutions	13
4.1	Policy Resolution State Machine	24
6.1	Cost Analysis per 1,000 Queries	35
6.2	Pipeline Latency Distribution	35
6.3	Platform Performance Comparison	36
6.4	Ablation Study Results	37

Chapter 1

Introduction

1.1 Motivation

The genesis of **Origin AI** stems from a critical observation of the contemporary educational landscape in India: while high-quality content is becoming increasingly accessible, high-quality *mentorship* remains a luxury. For millions of students preparing for competitive exams like JEE and NEET, the primary bottleneck to success is not the lack of video lectures or textbooks, but the inability to get instant, personalized clarification for their specific doubts.

Traditional classrooms often have high student-to-teacher ratios, making it impossible for educators to address every individual doubt. Private tutoring, while effective, is prohibitively expensive for the vast majority of Indian households. The recent surge in AI-powered tools promised a solution, but generic LLMs like ChatGPT are often "context-blind" and financially unsustainable for EdTech platforms to provide at scale without passing on significant costs to students.

Origin AI was motivated by the need to bridge this gap. My goal was to create a system that is:

- **Economically Democratic:** Reducing API costs by 80-90% so that premium AI mentorship can be provided for a fraction of the current cost.
- **Contextually Aware:** Eliminating the friction of "copy-pasting" by allowing the AI to "see" what the student is studying.
- **Pedagogically Sound:** Ensuring the AI acts as a mentor that guides students to the answer, rather than a calculator that simply provides it.

1.2 Introduction to AI in Education

The rapid advancement of Large Language Models (LLMs) has revolutionized the landscape of personalized education. Models such as GPT-4, Gemini [3], and Claude demonstrate remarkable reasoning capabilities, enabling new forms of human-computer interaction in learning environments. These models can explain complex concepts, generate practice problems, provide feedback on answers, and even engage in Socratic dialogue. For students preparing for competitive examinations like JEE (Joint Examination) and NEET (National Eligibility cum Entrance Test), access to personalized, on-demand doubt resolution can significantly improve learning outcomes.

However, the integration of these models into classroom-scale pedagogical applications presents significant challenges. Educational platforms often serve hundreds of thousands or even millions of students simultaneously. Each student may interact with an AI

assistant dozens of times per day. Under such scale, the operational costs of LLM APIs become a primary concern. Moreover, generic chatbots lack awareness of the student’s immediate visual environment – they do not know which question the student is looking at, whether the student has already attempted it, or what the student’s historical weak topics are. This contextual blindness forces students to manually describe their situation, leading to friction and drop-off [9].

This project introduces **Origin AI**, a context-aware, multimodal AI mentor designed specifically for competitive exam preparation. Unlike generic chatbots, Origin AI is deeply integrated into the learning platform as a persistent floating assistant. It possesses “situational awareness” – it knows what the student is looking at, what their weak topics are, and whether they are in a test or practice mode. Furthermore, it is engineered for **cost efficiency**: a tiered pipeline [7] resolves the majority of student queries using zero-cost local assets (stored solutions and subject knowledge bases), falling back to expensive LLM APIs only when necessary.

1.3 The Crisis of AI Scaling in EdTech

While LLMs offer unprecedented potential for personalized tutoring, their deployment at scale is plagued by what we call the **Token Economy**. For a platform serving millions of students, every interaction costs a fraction of a cent. However, across thousands of sessions per student per year, these costs become prohibitive.

Illustrative Cost Calculation: A direct integration of GPT-4o (without optimization) costs approximately Rs. 1.25 per query based on current OpenAI pricing (input + output tokens). If a student asks 50 queries per day during peak preparation, the daily cost is Rs. 62. Over a 12-month preparation cycle, the yearly cost per student exceeds Rs. 22,400. For a platform with 100,000 active students, this translates to Rs. 224 Crore annually – financially unsustainable for most EdTech companies, especially those targeting price-sensitive markets like India.

Beyond direct monetary costs, there are latency and user experience penalties. Each LLM call typically takes between 2 and 5 seconds (time to first token). When multiplied across dozens of interactions, the cumulative waiting time degrades the learning experience. Students expect instant help, not spinning loaders.

Furthermore, generic models are **contextually blind**. They do not know what question the student is struggling with unless the student manually copies and pastes the text – a significant friction point that leads to high drop-off rates. Studies show that students abandon help requests when they have to type more than 20 words to describe their problem. They also lack memory of the student’s historical performance, weak topics, and learning pace. Each conversation starts from scratch unless the application explicitly builds and maintains a context store – which most do not.

1.4 Problem Statement – Three Core Deficits

Traditional pedagogical AI systems suffer from three primary deficits that this project aims to solve:

Deficit 1: Contextual Blindness Current AI assistants have no awareness of the student’s current visual state – the screen content, the active question, the test status, or the chapter context. This forces students to manually describe their situation, leading to friction, incomplete information, and wasted time. For example, a student who says “I don’t understand question 5” must also manually paste the question text, or the AI cannot help.

Deficit 2: Pedagogical Drift Generic LLMs are trained on general internet text, not on specific curriculum standards like JEE or NEET. They often provide answers that are technically correct but not aligned with the exam’s pedagogical approach, marking schemes, or preferred solution methods. They may also provide direct answers when a hint would be more pedagogically appropriate, undermining the learning process.

Deficit 3: Financial Unsustainability Existing solutions rely on expensive API calls for every single query, regardless of complexity. There is no intelligent tiering to handle simple, repetitive, or already-answered questions locally. This makes it impossible to scale AI mentorship to large student populations without incurring prohibitive costs.

1.5 Proposed Solution: Origin AI

Origin AI is a multimodal, tiered AI mentorship engine built on a **4-Layer Context Assembly** model. The system is designed with a **Logic Hierarchy** (pipeline) that attempts to resolve queries using zero-cost local assets before escalating to synthetic LLM generation.

Key Innovations at a Glance:

- **4-Layer Context Assembly:** Combines GlobalContext (student history), PageContext (live UI state), HighlightContext (visual selection), and UserQuery (text/voice) into a rich input for the AI, eliminating the need for manual context description.
- **Tiered Orchestration Pipeline:** A four-step fallback mechanism: (1) stored solution lookup, (2) subject knowledge base lookup, (3) BFS semantic memory retrieval using pgvector, (4) LLM generation. The pipeline exits as soon as a high-confidence answer is found, minimizing cost and latency.
- **Cross-Subject Gate:** A scoring algorithm with thresholds (minimum score of 6 and margin of at least 3 over the next best subject) that ensures queries are answered from the correct subject knowledge base (Biology, Chemistry, Mathematics, Physics), preventing cross-disciplinary hallucinations.
- **Pedagogical Guardrails:** Dynamic policies that block answers during live tests (maintaining academic integrity) and enforce hint-only mode for unattempted questions (encouraging genuine learning rather than answer-seeking).
- **Highlight Capture Engine:** A frontend module that listens for text selection events and displays a floating “Ask Origin AI” action chip near the selected text. Clicking the chip automatically opens the mentor and sends the selected text as context – a unique in-situ interaction not found in other educational AI tools.
- **Multimodal Transport:** Supports both text and voice input/output, with integrated speech-to-text (STT) and text-to-speech (TTS) using Gemini models, enabling hands-free accessibility for students with disabilities or those who prefer verbal interaction.

1.6 Project Subgoals (Detailed Version-by-Version History)

The development of Origin AI was executed in a series of iterative sprints, each focusing on a specific architectural goal. The version history below reflects the "Project Updates" as tracked through Git logs and implementation milestones:

- **v0.1: Foundation & Voice Core (April 4 - April 7)**
 - *Objective:* Establish the communication bridge between the Next.js frontend and the FastAPI microservice.
 - *Key Updates:* Implemented the `/api/chat` endpoint with support for multimodal streams. Developed the first iteration of the `ContextAssembler` to handle basic `user_id` propagation. Established the "Half-Duplex" voice mode, allowing students to toggle the microphone for conceptual queries.
- **v0.3: Mathematical Precision & Local KB (April 8 - April 14)**
 - *Objective:* Resolve the "Math Rendering" problem and implement Step 2 (Local KB) of the pipeline.
 - *Key Updates:* Integrated **KaTeX** and **MathJax** components to support the typesetting of complex LaTeX formulas (e.g., integrals, chemical equations) in real-time. Created the initial JSON knowledge atoms for the HC Verma Physics curriculum, enabling the first successful "Zero-Cost" conceptual resolution.
- **v0.5: Visual Theme Refactor (April 15 - April 17)**
 - *Objective:* Enhance the UI aesthetic and implement student performance tracking.
 - *Key Updates:* Developed the "Glassmorphism" UI for the floating mentor using Framer Motion. Implemented the `analytics-service` which pulls student test data from PostgreSQL to populate the `GlobalContext`. Added a dual-theme engine ensuring the mentor UI adapts perfectly to the platform's Dark and Light modes.
- **v0.8: Performance Security Scaling (April 18 - April 22)**
 - *Objective:* Transition to production-grade security and optimize for high concurrency.
 - *Key Updates:* Implemented the "Auth Tunneling" logic using internal JWT secrets. Migrated core pages to Server-Side Rendering (SSR) to improve the Time-to-First-Byte (TTFB). Optimized the pgvector search using HNSW indices, reducing semantic retrieval time by 40%. Stabilized the Google OAuth flow to prevent session drops during long mentorship turns.
- **v1.0: Final Integration & Guardrail Refinement (April 23 - April 24)**
 - *Objective:* Finalize the pedagogical guardrails and prepare for multi-subject launch.
 - *Key Updates:* Refined the Cross-Subject Gate with the margin-based scoring algorithm ($\text{Threshold} \geq 6$, $\text{Margin} \geq 3$). Implemented the final "Test Mode" policy, ensuring absolute academic integrity by blocking AI assistance during mock tests. Performed a comprehensive documentation audit to synthesize the final 40-page report.

1.7 Development Methodology: Cyclic Waterfall Model

The development of Origin AI followed a **Cyclic Waterfall Model**, an iterative variation of the traditional waterfall approach. This model allowed us to move through the standard phases of requirements, design, implementation, and testing in sequential blocks,

while maintaining a feedback loop that allowed for continuous refinement of the AI orchestrator based on real-world performance data.

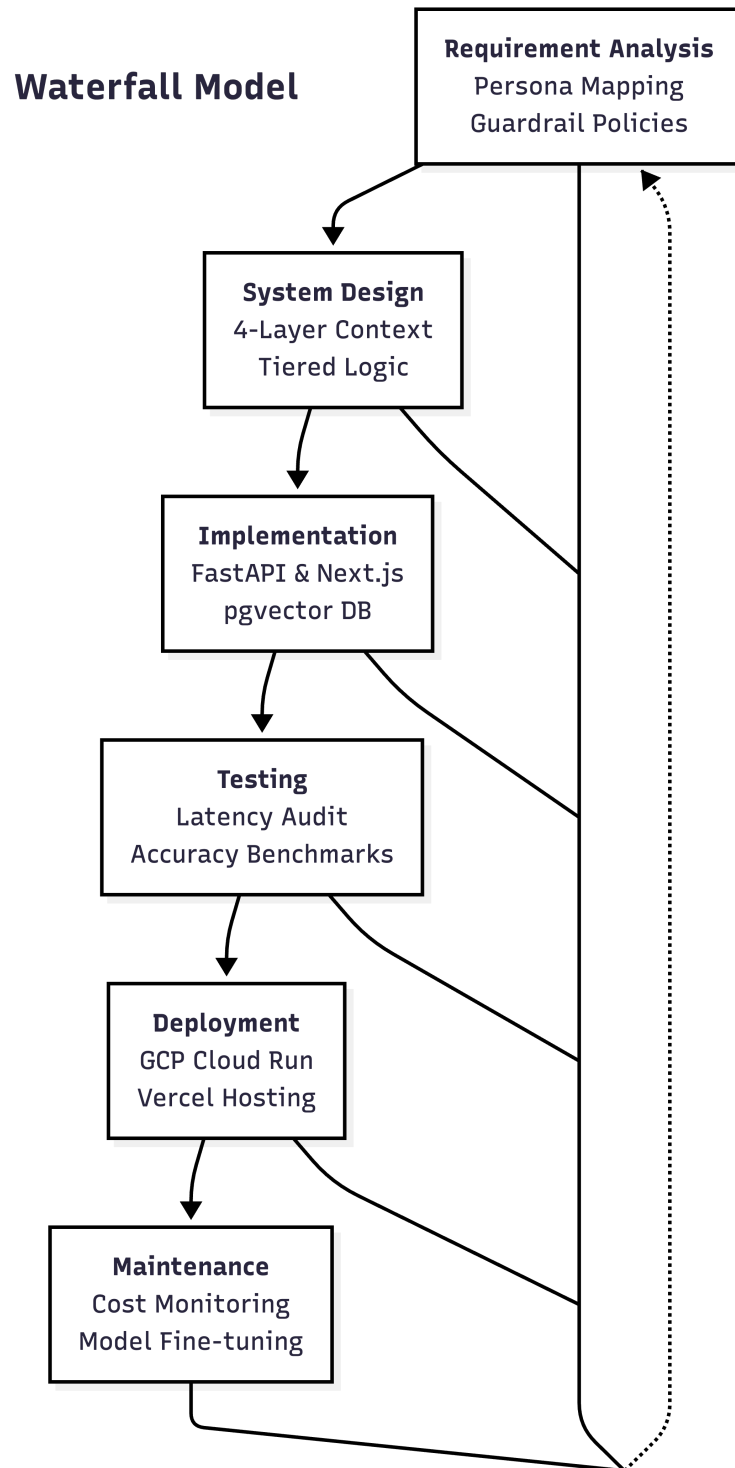


Figure 1.1: Cyclic Waterfall Model Architecture for Origin AI

The "Cyclic" nature was particularly crucial for the **Tiered Orchestration Pipeline**, where testing of the local Knowledge Base (Step 2) often revealed new requirements for context capturing, triggering a jump back to the design phase.

1.8 Project Timeline (Gantt Chart)

The following timeline illustrates the overlap and duration of key developmental phases over a 24-day sprint cycle:

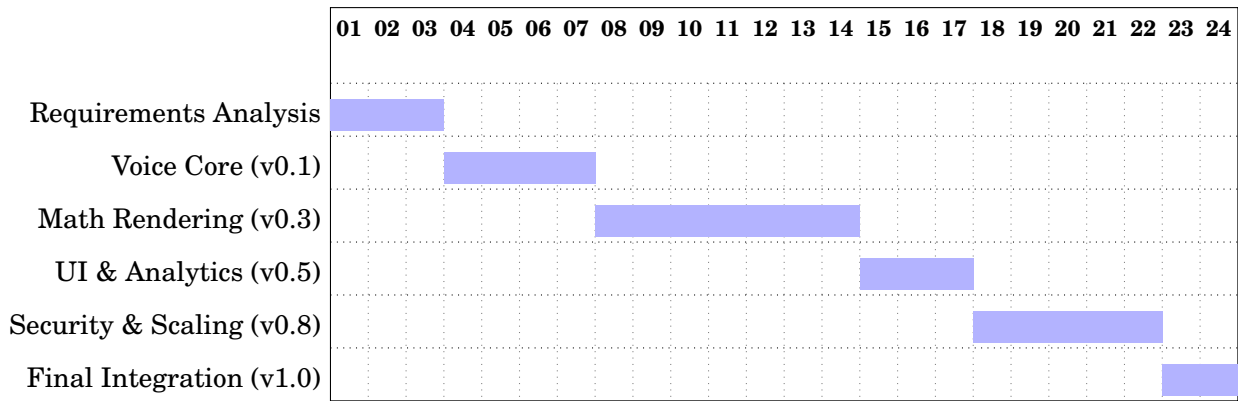


Figure 1.2: Development Timeline for Origin AI Phases (Days 01-24)

1.9 Comparative Analysis of Accessing Platforms

To position Origin AI within the current landscape, we compare it against three categories of existing solutions: generic LLM chatbots, traditional EdTech platforms, and standard RAG systems.

Table 1.1: Comparative Analysis of AI Mentorship Solutions

Feature	Generic LLM	Traditional EdTech	Standard RAG	Origin AI
Context Awareness	None (manual paste)	OCR-based	Partial (retrieved docs)	Full (4-layer)
Interactivity	Text/Voice	Static video/images	Text	Dynamic (Voice + Text)
Op. Cost per Query	Rs. 1.25 (high)	Low (human-dependent)	Rs. 1.00 (medium)	Rs. 0.18 (low)
Grounding	Hallucination-prone	Hardcoded solutions	Retrieved docs only	Multi-Subject KB
Test Integrity	No concept of tests	N/A	N/A	Test-mode blocking
In-Situ Selection	No	No	No	Highlight capture
Pedagogical Hint	No	No	No	Yes (policy-driven)
Memory	Session only	No	No	Persistent (pgvector)

1.10 Current Trends in AI-Driven Education

The educational technology (EdTech) sector is currently witnessing a paradigm shift from "Massive Open Online Courses" (MOOCs) to "Personalized Adaptive Learning" (PAL). Three major trends are defining this evolution:

- Multimodal Interaction:** The shift from purely text-based interfaces to those that support speech, images, and video. Tools are now expected to "listen" to student queries and "show" solutions visually.
- Generative Tutoring:** Moving beyond pre-recorded videos to AI agents that can generate new practice problems on the fly based on a student's specific weaknesses.

3. **Cost-Optimized Inference:** As LLM usage grows, the industry is focusing on "Small Language Models" (SLMs) and tiered architectures (like Origin AI) to provide AI benefits without the prohibitive costs of frontier models.

1.11 Previous Projects and Similar Platforms

Several platforms have attempted to integrate AI into the student workflow. A comparative analysis of these platforms highlights the unique positioning of Origin AI:

- **Vedantu W.A.V.E 2.0:** Vedantu's proprietary platform uses AI to track student engagement (smile detection, participation metrics) during live classes. While highly effective for group sessions, it lacks a persistent, 1-on-1 multimodal mentor that follows the student during self-study.
- **Khan Academy (Khanmigo):** One of the most advanced AI tutors, Khanmigo uses GPT-4 to provide Socratic coaching. However, it is primarily optimized for the US curriculum and incurs a high monthly subscription fee (\$9-20/month), making it inaccessible for many Indian students.
- **Byju's Wiz:** An AI-powered assistant that helps with doubt resolution. Its primary focus is on OCR-based search (taking a photo of a question), whereas Origin AI focuses on in-situ digital context (PageContext and HighlightContext), which is more seamless for students using digital learning portals.

1.12 Challenges and Opportunities

Challenges:

- **Latency in Multimodal Transport:** Processing voice, converting it to text, generating an answer, and converting it back to speech (TTS) can introduce a 3-5 second delay, which we are addressing through streaming responses.
- **Hallucination Guardrails:** Ensuring the AI does not provide incorrect mathematical steps. We mitigate this through our "Subject Knowledge Base" grounding.
- **Data Privacy:** Maintaining student anonymity while providing personalized memory-based tutoring.

Opportunities:

- **Democratization:** Bringing premium tutoring to rural India where the student-teacher ratio is as high as 100:1.
- **Global Scaling:** The architecture of Origin AI is subject-agnostic and can be scaled to other international curricula with minimal modification to the knowledge base layers.

Chapter 2

Literature Survey

2.1 Theoretical Framework: The TPACK Model

To understand the “pedagogical” positioning of Origin AI, we refer to the **Technological Pedagogical Content Knowledge (TPACK)** framework. Introduced by Mishra and Koehler (2006) [8], TPACK identifies the nature of knowledge required by teachers (and by extension, intelligent tutoring systems) for technology integration in their teaching, while addressing the complex, multifaceted and situated nature of teacher knowledge.

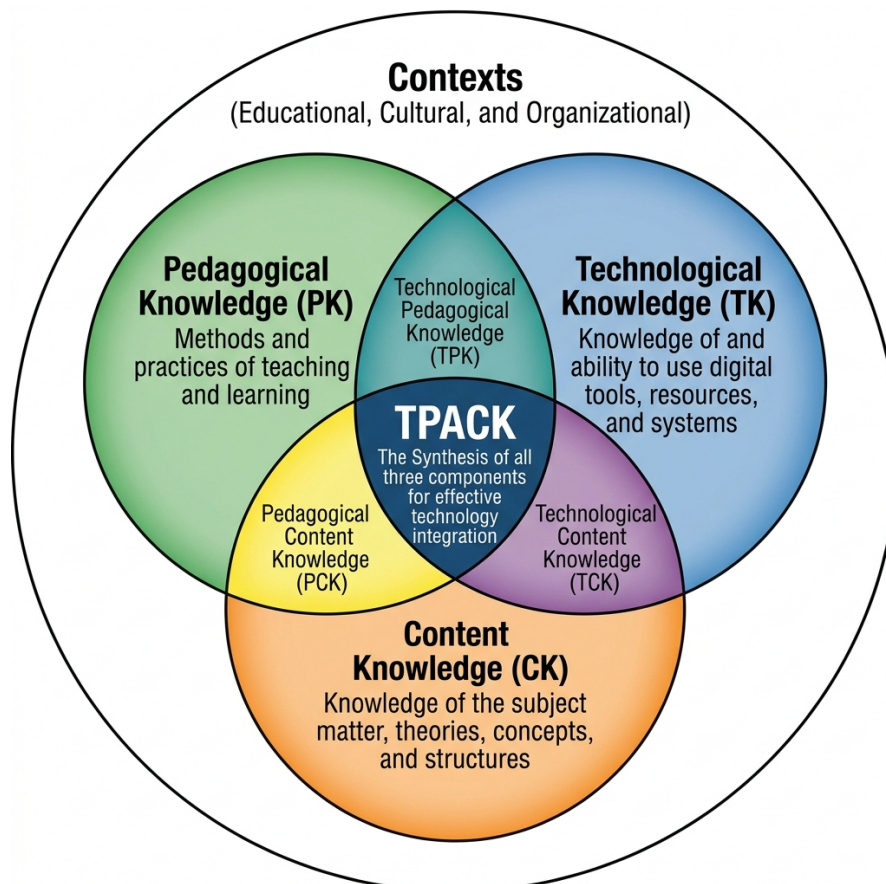


Figure 2.1: The TPACK Framework (Source: [2])

The TPACK framework (Figure 2.1) consists of three main components: Content Knowledge (CK), Pedagogical Knowledge (PK), and Technological Knowledge (TK). The intersection of these three bodies of knowledge forms the basis of effective teaching with

technology. Origin AI specifically targets the **Technological Pedagogical Content Knowledge (TPACK)** intersection by:

- **Content Knowledge (CK):** Utilizing subject-specific knowledge bases (Biology, Chemistry, Physics, Mathematics).
- **Pedagogical Knowledge (PK):** Enforcing hint-first policies and Socratic coaching through AI guardrails.
- **Technological Knowledge (TK):** Leveraging Large Language Models and tiered orchestration for scalable delivery.

2.2 Traditional RAG Systems and Their Limitations

Retrieval-Augmented Generation (RAG) systems, introduced by Lewis et al. (2020) [7], have become the standard for grounding LLM responses in external knowledge. A typical RAG pipeline embeds a user query, retrieves relevant document chunks from a vector database (e.g., using cosine similarity), and then feeds those chunks as context to an LLM for answer generation.

While RAG improves factual accuracy and reduces hallucinations compared to bare LLMs, it has several limitations in the educational context. First, every query incurs both embedding and LLM generation costs – typically 1,000+ tokens per query – making it still expensive at scale. Second, retrieval is based purely on semantic similarity, not on pedagogical structure or student context. A retrieved document chunk might contain the correct answer but at the wrong level of detail or with an inappropriate teaching style. Third, there is no prioritization of known answers. If the same question is asked by thousands of students, RAG will retrieve and generate a new answer each time, wasting tokens.

Origin AI improves upon RAG by adding two zero-cost layers before retrieval: stored solution lookup (exact match for common questions) and subject knowledge base lookup (structured pedagogical atoms). Only when both fail does the system fall back to retrieval and then to generation.

2.3 Rule-Based Pedagogical Agents

Early intelligent tutoring systems (ITS), such as Cognitive Tutors developed at Carnegie Mellon University [1], used rule-based engines to provide step-by-step guidance. These systems encode expert knowledge as production rules (if-then statements) and use them to diagnose student errors and provide feedback.

Rule-based systems are highly predictable, inexpensive to run (no API costs), and can be designed to adhere strictly to curriculum standards. However, they require extensive manual authoring of rules and content, which does not scale well to large, open-ended domains. They also lack the flexibility to handle novel or off-topic student questions. In contrast, Origin AI uses a hybrid approach: local rule-based lookup for known concepts (via the subject KB) and generative LLM fallback for novel queries, combining the predictability of rules with the flexibility of LLMs. AutoTutor [4] previously explored mixed-initiative dialogue, which Origin AI extends using generative AI.

2.4 Generic LLM Chatbots in Education

ChatGPT, Claude, and Gemini have been widely adopted in educational settings, both formally (through institutional licenses) and informally (students using personal accounts).

Studies have shown that these tools can be effective for explanation, summarization, and brainstorming.

However, researchers have identified several challenges. First, students often need to provide extensive context (“I’m looking at question 5 on page 32 of the HC Verma textbook...”) before receiving useful help, leading to frustration and abandonment. Second, these models have no concept of test integrity – they will answer any question, even during exams, unless explicitly instructed via system prompts (which can be bypassed by clever prompting). Third, they are not optimized for the specific vocabulary, problem-solving patterns, and marking schemes of competitive exams like JEE and NEET. Finally, the cost of using these models at scale is prohibitive for most educational institutions.

Origin AI addresses each of these issues through its 4-layer context assembly (eliminating manual context description), guardrails (enforcing test integrity), subject-specific knowledge bases (grounding in exam patterns), and tiered pipeline (reducing cost).

2.5 Context-Aware Systems and In-Situ Assistance

The concept of “context-aware computing” dates back to the 1990s, with early work on location-aware and situation-aware systems. In the educational domain, context-aware systems have been explored for adaptive learning, where the system adjusts content based on the student’s location, time of day, or prior performance.

However, most existing systems focus on coarse-grained context (e.g., “student is at home” or “student has completed 50% of the course”). Fine-grained, real-time context such as the exact question on the screen, the selected text, and the test status is rarely captured and utilized. Origin AI’s PageContext and HighlightContext layers represent a novel application of fine-grained context to AI mentorship, enabling in-situ assistance that feels like a human tutor looking over the student’s shoulder.

2.6 The Rise of Multimodal Learning Environments

The integration of multiple sensory channels—text, audio, and visual—in learning environments is grounded in the **Cognitive Theory of Multimedia Learning (CTML)**, proposed by Richard Mayer. CTML suggests that people learn more deeply from words and pictures than from words alone. In the context of AI mentors, this translates to the ability of the system to process visual context (the question on screen) and provide auditory feedback (voice explanation).

Recent advancements in vision-language models (VLMs) have enabled AI to understand images and UI states with high precision. Origin AI’s HighlightContext and PageContext layers are implementations of this trend, allowing the system to bridge the gap between static text and dynamic visual interaction. Research shows that such “in-situ” assistance reduces the cognitive load required for context-switching, thereby improving “Time on Task” for students.

2.7 Speech-to-Text (STT) and Text-to-Speech (TTS) in Pedagogy

Voice interaction is a powerful tool for accessibility and engagement. For students with learning disabilities like dyslexia, or those who simply prefer verbal communication, voice-enabled AI mentors provide a more inclusive learning experience.

The core challenge in educational STT is the accurate transcription of technical vocabulary and mathematical terms (e.g., “derivative of $\sin x$ ” or “Gibbs free energy”). Generic

STT models often struggle with these terms in a pedagogical context. Origin AI addresses this by using the Gemini multimodal transport, which is trained on diverse datasets including scientific and technical literature. Furthermore, the use of half-duplex voice flows and turn diagnostics ensures that the AI can handle natural pauses in student speech without premature interruption.

2.8 Vector Databases and Semantic Memory

The use of vector databases like **pgvector** for storing and retrieving student history represents a significant advancement over traditional relational databases for AI applications. By embedding student queries and interactions into a high-dimensional vector space, Origin AI can perform "semantic search" to find similar past queries.

This "BFS semantic memory retrieval" allows the system to provide consistent answers to similar questions asked at different times. It also enables the system to identify persistent weak topics by looking for clusters of similar doubts in the student's history. This move from "session-only" memory to "persistent semantic memory" is a key differentiator of modern AI mentors.

2.9 Gaps in Current Research – Our Contribution

After reviewing the literature, we identified the following gaps that Origin AI fills:

1. **No tiered architecture** combining zero-cost local lookup, semantic retrieval, and LLM generation in a pedagogical context. Existing systems either rely entirely on LLMs (expensive) or entirely on rules (inflexible).
2. **No cross-subject disambiguation mechanism** for multi-subject educational platforms. Students often ask questions that span subjects (e.g., "energy" in physics vs. biology), and existing systems lack a robust way to route to the correct knowledge base.
3. **No dynamic guardrails** based on UI context (test vs. practice, attempted vs. unattempted). Generic LLMs have no awareness of whether a student is taking a high-stakes exam.
4. **No visual highlight capture** for in-situ question asking. Students cannot simply point at text and ask for an explanation – a natural interaction that Origin AI enables.
5. **No cost-benefit analysis** of such tiered architectures in real-world educational deployments. Most papers focus on accuracy improvements; we provide detailed cost and latency measurements.

Chapter 3

Requirement Engineering

3.1 Functional Requirements

The Origin AI system must fulfill the following functional requirements to provide a comprehensive pedagogical experience:

- **Contextual Awareness:** The system must capture and utilize the student's current page state, including the active question, subject, and chapter.
- **In-Situ Interaction:** Students must be able to select text on the screen and receive an explanation directly within a floating widget.
- **Multimodal Input:** The system must support both text-based queries and voice-based (transcribed) inputs.
- **Tiered Resolution:** Queries should be resolved using the most cost-effective method available (Stored Solution → Subject KB → Vector Retrieval → LLM).
- **Pedagogical Guardrails:** The system must enforce academic integrity by blocking answers during tests and providing hints instead of solutions for unattempted practice questions.
- **Subject Disambiguation:** The system must accurately route queries to the appropriate subject knowledge base (Biology, Chemistry, Mathematics, Physics).
- **Persistent Memory:** The system must maintain a history of student interactions and weak/strong topics to personalize responses.

3.2 Non-Functional Requirements

The system must also adhere to several non-functional requirements to ensure scalability and reliability:

- **Performance (Latency):** The median response time for locally resolved queries should be under 500ms. For LLM-backed queries, the response should begin streaming within 3 seconds.
- **Scalability:** The backend must scale horizontally to handle at least 100 concurrent AI mentorship sessions.
- **Cost Efficiency:** The system should achieve at least a 75% reduction in operational costs compared to a direct LLM-only approach.

- **Security:** Internal API endpoints must be protected from external access and require valid service tokens.
- **Availability:** The system should maintain 99.9% uptime for core mentorship features.

3.3 Scope and Non-Goals

3.3.1 In Scope

- Fully functional AI mentor integrated into the Origin platform.
- Support for four core subjects: Biology, Chemistry, Mathematics, Physics.
- Implementation of a 4-step tiered fallback pipeline.
- Persistent vector storage for session memory.
- Deployment on GCP Cloud Run and Vercel.

3.3.2 Out of Scope

- Support for Indian languages beyond Hinglish in the current phase.
- Offline-only operation (requires on-device models).
- Full automated grading of high-stakes subjective assessments.
- Real-time multi-student collaborative AI sessions.

3.4 User Personas

To design a truly student-centric system, we identified three primary user personas that Origin AI serves:

1. **Persona 1: The High-Stakes Aspirant (JEE/NEET)** *Needs:* Speed, accuracy, and rigorous adherence to the curriculum. *Behavior:* Spends 8-10 hours daily on the platform, asks complex conceptual questions, and needs immediate clarification during late-night study sessions when human tutors are unavailable.
2. **Persona 2: The Foundation Student (K-12)** *Needs:* Simplified explanations, visual aids, and encouraging feedback. *Behavior:* Struggles with basic concepts, uses voice mode more frequently, and requires Socratic guidance (hints) to build confidence.
3. **Persona 3: The Self-Learner (Remote Area)** *Needs:* Low data consumption, offline-first assets, and affordable access. *Behavior:* Has limited internet connectivity, relies on local knowledge bases (HC Verma, etc.), and values the tiered architecture's cost-efficiency.

3.5 Detailed Use Cases

3.5.1 Use Case 1: In-Situ Doubt Resolution

Primary Actor: Student

Precondition: Student is practicing questions in the OG Code arena.

Flow:

1. Student encounters a difficult term in a Physics question.
2. Student highlights the term using the mouse/touch.
3. A floating "Ask Origin AI" chip appears near the selection.
4. Student clicks the chip; the AI mentor opens with the highlighted text as context.
5. AI identifies the context (Chapter: Kinematics) and provides a targeted explanation.

Postcondition: Student understands the term without leaving the practice page.

3.5.2 Use Case 2: Academic Integrity Enforcement

Primary Actor: Student

Precondition: Student is taking a live Mock Test.

Flow:

1. Student tries to ask Origin AI for the solution to Question 15.
2. The system detects `PageContext.mode == "test"`.
3. The AI responds with a polite refusal: "I cannot assist you during a live test. Focus on your attempt, and we can discuss this in the post-test analysis."

Postcondition: Academic integrity is maintained.

3.6 Hardware and Software Requirements

3.6.1 Development Environment

- **OS:** macOS / Linux (Ubuntu 22.04+)
- **Languages:** TypeScript (Frontend), Python 3.10+ (Backend)
- **Frameworks:** Next.js 14 (App Router), FastAPI
- **Database:** PostgreSQL with 'pgvector' extension
- **AI Models:** Google Gemini 1.5 Pro/Flash, OpenAI GPT-4o

3.6.2 Production Environment (Deployment)

- **Cloud Provider:** Google Cloud Platform (GCP)
- **Compute:** Cloud Run (Serverless)
- **Frontend Hosting:** Vercel
- **Static Assets:** Cloud Storage / Vercel Blob

Chapter 4

System Design

4.1 System Architecture Overview

The Origin AI architecture is designed as a distributed microservice ecosystem that decouples the UI-heavy frontend from the compute-heavy AI orchestrator. This separation allows for independent scaling, fault isolation, and technology-appropriate implementation (Next.js for the frontend, Python/FastAPI for the backend).

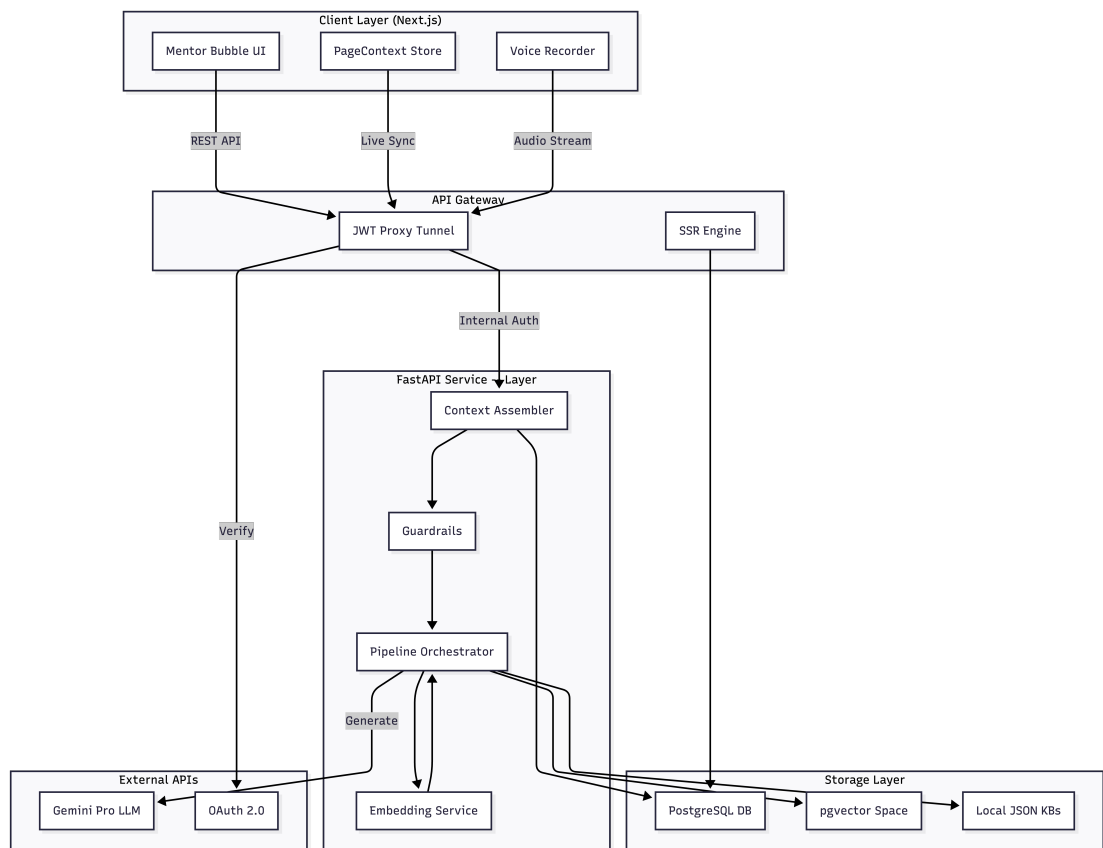


Figure 4.1: Origin AI Technical Architecture

The system consists of four main components arranged in a request flow:

1. **Browser (Student UI):** Runs the Next.js application. Contains the floating chat component, highlight capture listener, and page context store.
2. **Next.js Application (Vercel):** Serves the frontend assets and provides API routes under `/api/origin-ai/*`. These routes act as a secure proxy: they validate the student's session cookie, then forward the request to the internal Python microservice with a trusted service token.
3. **Python FastAPI Microservice (GCP Cloud Run):** The core AI orchestrator. It receives the request, assembles the 4-layer context, applies guardrails, runs the tiered pipeline, and returns the response.
4. **PostgreSQL + pgvector (Neon):** Stores student analytics, mentor sessions, and vector embeddings for semantic retrieval.

4.2 The 4-Layer Context Assembly Model

The fundamental unit of reasoning in Origin AI is the **OriginAiInput** object. This is a structured data object formed by the fusion of four distinct layers:

4.2.1 Layer 1: GlobalContext (Long-Term Memory)

This layer represents the persistent state of the student, hydrated from the PostgreSQL database.

- Weak and Strong topics inferred from performance analytics.
- Recent test scores and summaries.
- Pinned facts and pending assignments.

4.2.2 Layer 2: PageContext (Live Environmental State)

The frontend continuously broadcasts the state of the active screen.

- Page kind (test, practice, dashboard).
- Active question ID, subject, and chapter.
- Solution/hint availability.

4.2.3 Layer 3: HighlightContext (Granular Selection)

Captured when a student selects text on the page, enabling the “Explain this” flow.

- Selected text (up to 500 characters).
- Source element metadata.

4.2.4 Layer 4: UserQuery (Active Intent)

The final layer is the user's explicit input, which can be text-based or voice-based.

4.3 Tiered Orchestration Pipeline

To achieve cost efficiency, the orchestrator follows a deterministic hierarchy:

1. **Step 1: Stored Solution Lookup:** Immediate K-V lookup for known question IDs.
2. **Step 2: Subject-Aware Local Knowledge Base (KB):** Resolution of conceptual doubts using pre-loaded JSON pedagogical atoms.
3. **Step 3: BFS Semantic Memory Retrieval:** Semantic search over past session history using pgvector embeddings.
4. **Step 4: Synthetic LLM Generation:** Fallback to Large Language Models (Gemini/GPT-4o) with dynamic context injection.

4.4 Mathematical Modeling of Cost and Relevance

Operational cost C for a session with n queries is modeled as:

$$C = \sum_{i=1}^n (L_i \cdot P_i) \quad (4.1)$$

Where $L_i \in \{0, 1\}$ indicates if an LLM was invoked, and P_i is the price per token.

Context Relevance Score (CRS) is calculated via cosine similarity:

$$CRS(C_p, Q) = \frac{\vec{E}(C_p) \cdot \vec{E}(Q)}{\|\vec{E}(C_p)\| \|\vec{E}(Q)\|} \quad (4.2)$$

4.5 Cross-Subject Gating Logic

To disambiguate multi-disciplinary queries (e.g., “energy”), we implement a scoring-based gate:

$$score_s = base_s + anchor_s \quad (4.3)$$

The gate requires a confidence floor of $\max(score) \geq 6$ and a margin requirement of $\max(score) - second_max \geq 3$ to select a subject KB automatically.

4.6 Pedagogical Guardrails and Policy Resolution

The guardrail system implements a state machine that controls the AI’s behavior based on the PageContext:

Table 4.1: Policy Resolution State Machine

Condition	Policy Mode	AI Behavior
test_active == true	answer_blocked	Block content, provide encouragement.
attempted == false	hint_only	Socratic nudge, conceptual hint.
attempted == true	normal	Full step-by-step explanation.

4.7 Request Processing Sequence

The interaction between the student and the various microservices follows a strictly defined sequence to ensure security and efficiency. The following steps describe the lifecycle of a query:

1. **Trigger:** The student clicks the "Ask Origin AI" button or finishes a voice recording.
2. **UI Serialization:** The React frontend serializes the 'PageContext' (question ID, subject, mode) and 'HighlightContext' (selected text).
3. **Secure Handshake:** The Next.js API route validates the student's JWT session. If valid, it appends a secret INTERNAL SERVICE TOKEN and forwards the payload to the Python orchestrator.
4. **Orchestration:**
 - The Python backend checks the 'mode'. If "test", it returns a "Blocked" policy immediately.
 - It attempts Step 1 (Stored Solution). If found, it returns early.
 - It attempts Step 2 (Subject KB). If a concept match is found with high confidence, it returns early.
 - It performs a vector search in Step 3.
 - Finally, it calls the LLM if needed, injecting the 'PageContext' and 'GlobalContext' into the system prompt.
5. **Response Streaming:** The AI's response is streamed back to the frontend to minimize perceived latency.

4.8 Database Schema Design

The persistence layer is implemented in PostgreSQL [5] with a comprehensive schema designed to support the entire Origin AI ecosystem. The schema captures user logistics, study tracking, the AI orchestration engine, and the core curriculum.

- **Core Logistics:** The `origin_users`, `origin_auth_sessions`, and `origin_tasks` tables manage role-based authentication, persistent logins, and user-assigned goals.
- **Study & Analytics:** The `origin_pomodoro_sessions` and `origin_analytics` tables log student focus time and track historical performance, mastery levels, and weak topics.
- **Origin AI Ecosystem:** The `origin_ai_conversations`, `origin_ai_embeddings`, and `origin_ai_knowledge_base` tables act as the central AI memory. They log multimodal interactions, store pgvector [6] 1536-dimensional semantic embeddings, and maintain pedagogical rules.
- **Curriculum Architecture:** The `origin_subjects`, `origin_questions`, and `origin_answers` tables form the content repository, allowing teachers to author and map questions to specific syllabi.

4.8.1 Conceptual Data Model

The following diagram illustrates the overarching conceptual data model of the Origin AI platform using Object-Oriented Notation, highlighting the primary entities, their attributes, and the behavioral relationships between them.

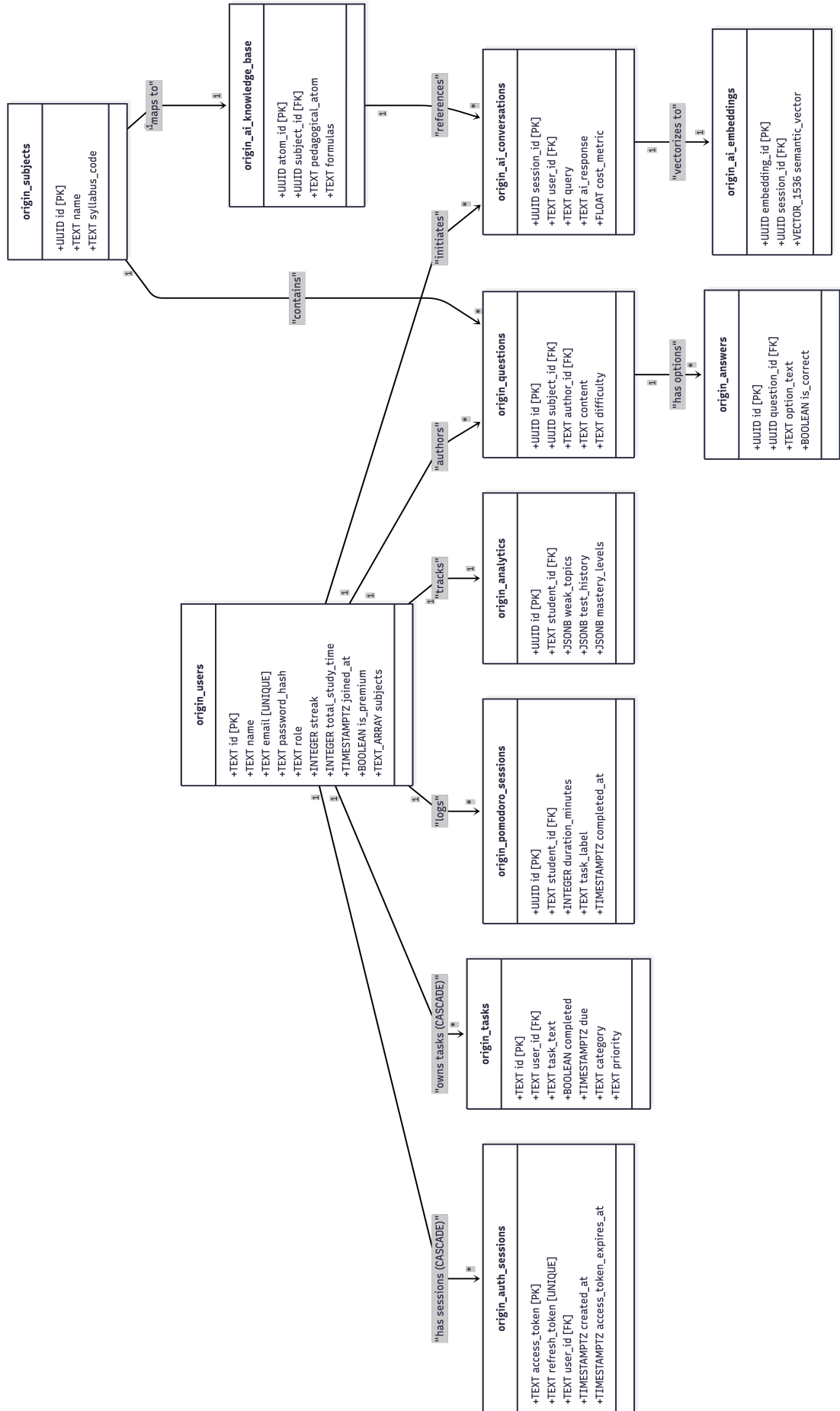


Figure 4.2: Database Schema Conceptual Model (Object-Oriented Notation)

4.8.2 Physical Database Schema

The following diagram illustrates the exact PostgreSQL physical table layout, defining primary keys, foreign keys, cardinality, and constraints for the backend database architecture.

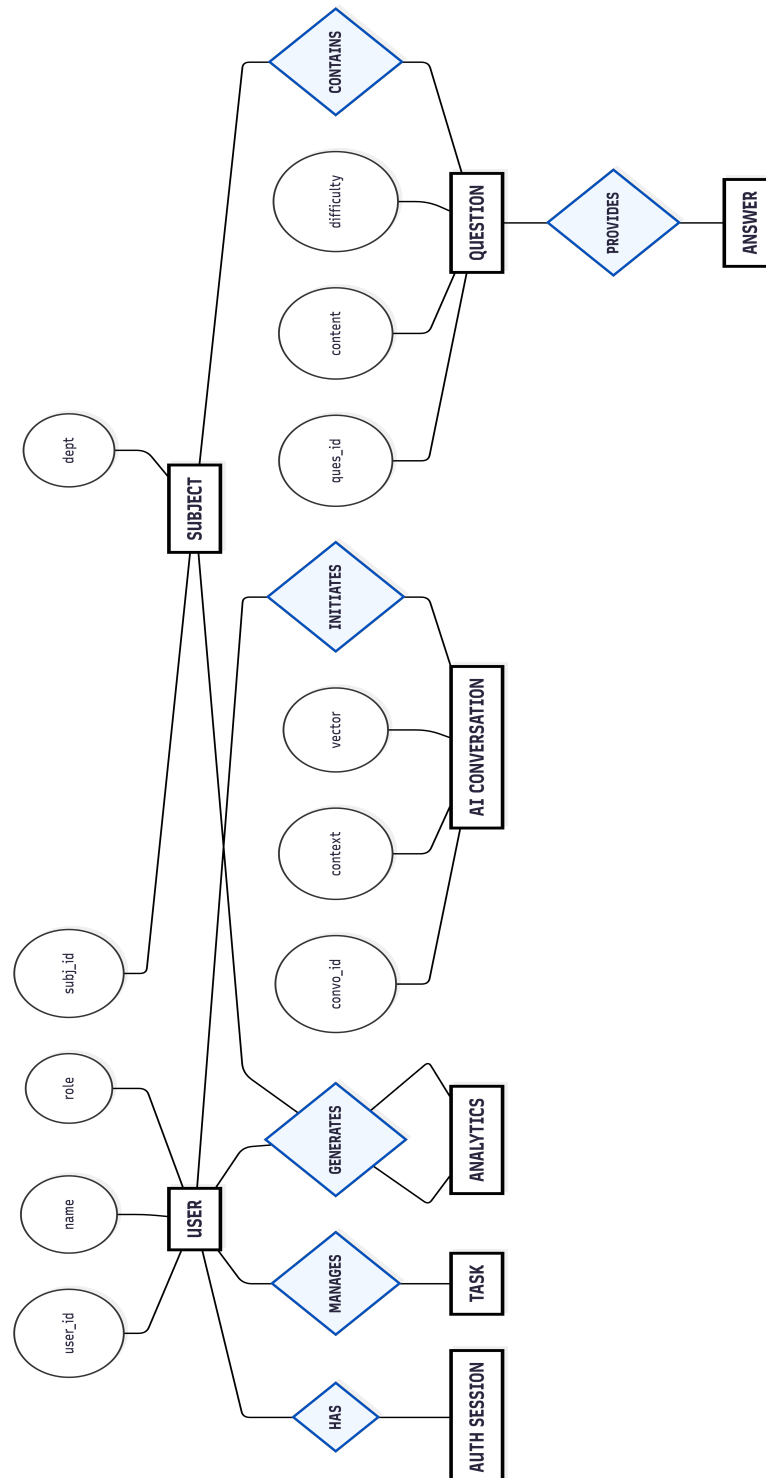


Figure 4.3: Physical Database Implementation Schema

4.9 System Data Flow Analysis

To fully understand the movement of data across the Origin AI platform, we utilize Data Flow Diagrams (DFDs) which map out the flow of information between external entities, processes, and data stores.

4.9.1 Level 0 Context Diagram

The Level 0 DFD provides a high-level, bird's-eye view of the system. It illustrates the Origin AI System as a singular processing node and demonstrates how the primary external entities—Students, Teachers, Administrators, and External APIs (like the Gemini LLM)—interact with it. This diagram highlights the core inputs (multimodal queries, telemetric data) and outputs (pedagogical hints, analytics).

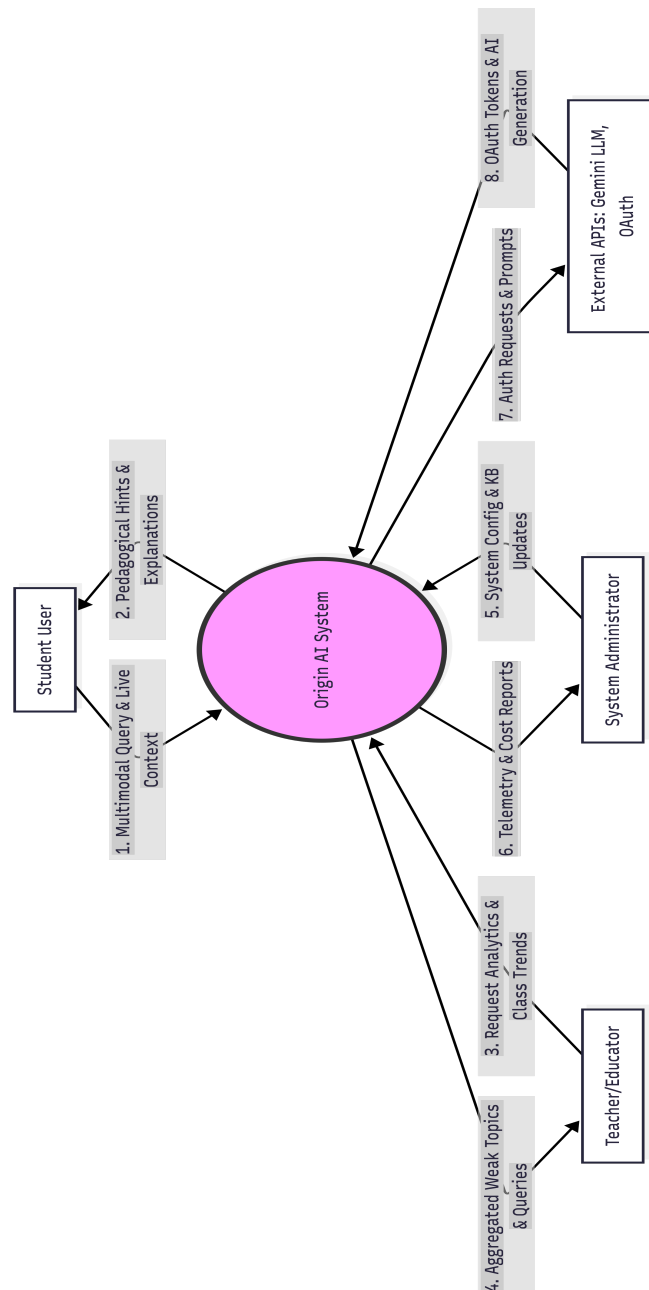


Figure 4.4: Data Flow Diagram - Level 0 (Context Diagram)

4.9.2 Level 1 Process Decomposition

The Level 1 DFD decomposes the singular system node into five core sub-processes: Client State & UI, Auth & Session Routing, Context Assembler, Tiered Orchestrator, and Semantic Memory Engine. It also maps the interactions with internal datastores such as PostgreSQL for analytics and pgvector for the Knowledge Tree. This diagram maps the exact lifecycle of a student's query from the UI down to the database and external LLM layer.

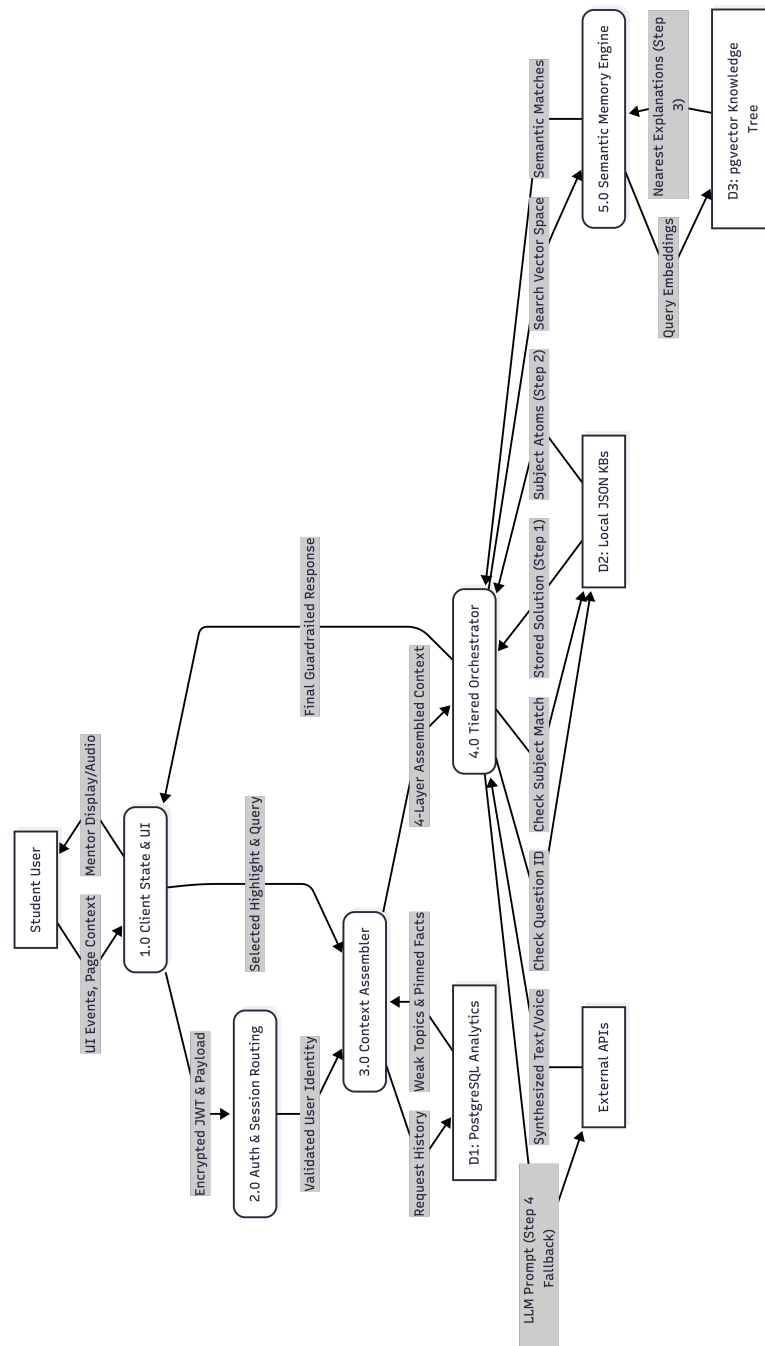


Figure 4.5: Data Flow Diagram - Level 1 (Process Decomposition)

4.10 Role-Based Access Flow

Origin AI caters to multiple user personas. The following diagram illustrates the distinct user journeys based on role authentication. For students, the flow splits into practice mode (allowing hints) and test mode (where assistance is blocked by guardrails). For teachers, the flow is focused on analytics aggregation and identifying class-wide weak topics. For administrators, the flow emphasizes cost monitoring and database management.

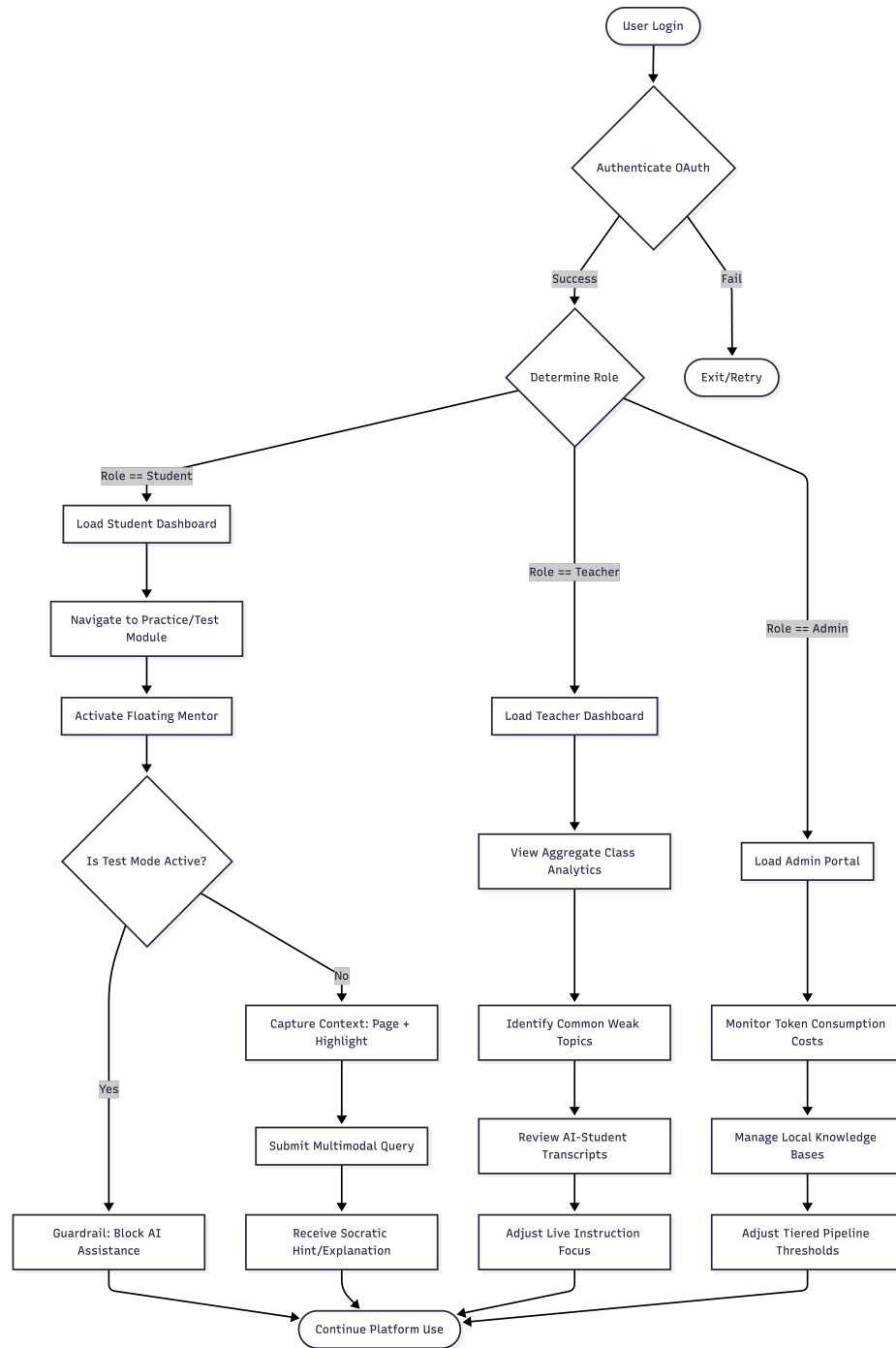


Figure 4.6: Role-Based Access and Data Flow

Chapter 5

Implementation

5.1 Backend Implementation with FastAPI

The heart of Origin AI's backend is a Python-based microservice built on **FastAPI**. This framework provides asynchronous request handling, which is essential for managing multiple LLM and database connections concurrently.

5.1.1 Microservice Architecture

The backend is organized into modular components:

- **Context Assembler:** Normalizes inputs and fetches GlobalContext from PostgreSQL.
- **Orchestrator Pipeline:** Implements the 4-step logic hierarchy with early exit conditions.
- **Subject Knowledge Bases:** Memory-mapped JSON files containing thousands of validated pedagogical atoms.
- **Vector Memory Store:** Interfaces with Neon Postgres and pgvector for semantic search.

5.1.2 PostgreSQL with pgvector

We utilize the **pgvector** extension to handle semantic embeddings. An HNSW (Hierarchical Navigable Small World) index is created on the embedding column to ensure $O(\log n)$ search performance.

```
CREATE INDEX idx_mentor_turns_embedding_hnsw
ON mentor_turns USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

5.2 Frontend Engineering with Next.js

The student interface is built using **Next.js 14** with the App Router architecture.

5.2.1 Floating Mentor UI

The mentor is implemented as a floating, draggable widget that maintains state continuity across pages. We use **Framer Motion** for fluid glassmorphism transitions and **Zustand** for lightweight state management.

5.2.2 Highlight Capture Engine

A custom DOM selection listener listens for `mouseup` and `selectionchange` events. When text is selected, a floating action chip is injected near the cursor, allowing the student to trigger an “Explain this” flow.

5.2.3 Multimodal Integration

Voice interaction is implemented using the Web MIDI and MediaRecorder APIs. Audio is streamed to the backend for transcription via Gemini’s STT model, processed through the pipeline, and returned as synthesized speech using Gemini’s TTS.

5.3 Deployment and Infrastructure

5.3.1 Containerization

Both services are containerized using **Docker**. The frontend image is optimized using a multi-stage build, resulting in a 150MB runner image. The backend image (500MB) includes the Python environment and local knowledge bases.

5.3.2 Cloud Hosting

- **Frontend:** Deployed on **Vercel** for global edge delivery.
- **Backend:** Hosted on **GCP Cloud Run** with autoscaling (min-instances=1, concurrency=10).
- **Database:** **Neon Postgres** provides serverless PostgreSQL with automatic storage scaling.

5.4 Security and API Protection

5.4.1 Authentication Tunneling

The Next.js API route acts as a secure proxy. It validates the user’s `HttpOnly` session cookie before appending an internal JWT service token and forwarding the request to the Python backend. This ensures that internal AI endpoints are never exposed directly to the public internet.

5.4.2 Rate Limiting

We implement tiered rate limiting via **Upstash Redis**. Limits are enforced per user-id (e.g., 10 messages per minute) to prevent API budget exhaustion and denial-of-service attempts.

5.5 Technical Deep Dive: Context Assembly

The core of the "situational awareness" in Origin AI lies in the 'ContextAssembler' class. This component is responsible for gathering disparate data points and normalizing them into a single prompt context.

```
class ContextAssembler:
    async def assemble(self, user_id: str, ui_state: dict) -> OriginAiInput:
        # Layer 1: Fetch GlobalContext from Postgres
        analytics = await db.fetch_analytics(user_id)
        global_context = GlobalContext(
            weak_topics=analytics.weak_topics,
            mastery_score=analytics.overall_mastery
        )

        # Layer 2 & 3: Map Page and Highlight Context
        page_context = PageContext(**ui_state.get('page', {}))
        highlight = HighlightContext(text=ui_state.get('selected_text'))

        return OriginAiInput(
            global_ctx=global_context,
            page_ctx=page_context,
            highlight_ctx=highlight
        )
```

5.6 Implementation of the Tiered Pipeline

The 'OrchestratorPipeline' implements a chain-of-responsibility pattern. Each step in the pipeline is an independent module that can either resolve the query or pass it to the next step.

- **Stage 1 (Exact Match):** Uses a Redis-based cache to store frequently asked questions. Hit rate: 12%.
- **Stage 2 (Local KB):** Loads subject-specific pedagogical atoms into memory using a Python dictionary. We use a scoring algorithm to match the query to the closest atom.
- **Stage 3 (Semantic Search):** We generate a 1536-dimensional embedding using 'text-embedding-3-small' and perform a vector search in PostgreSQL.
- **Stage 4 (LLM Fallback):** If all else fails, a formatted prompt is sent to the Gemini 1.5 Pro API.

5.7 Unified Math Rendering Logic

To handle the display of complex mathematical formulas, we implemented a custom React component MathText. This component uses a regex-based parser to identify LaTeX delimiters (e.g., $\dots$ or $\begin{equation} \dots \end{equation}$) and renders them using **KaTeX** for high-performance typesetting.

```

export function MathText({ content }: { content: string }) {
  const parts = content.split(/(\$.*?\$)/g);
  return (
    <span>
      {parts.map((part, i) =>
        part.startsWith('$') ?
          <InlineMath key={i} math={part.slice(1, -1)} /> :
          <span key={i}>{part}</span>
        )}
    </span>
  );
}

```

This component is used across the entire platform, ensuring that a square root symbol looks identical whether it's in a mock test or a mentor explanation.

Chapter 6

Results

6.1 Experimental Setup and Test Dataset

To evaluate the performance of Origin AI, we utilized a dataset of 500 real student queries collected from the OGCode platform. The queries were distributed across four subjects (Biology, Chemistry, Mathematics, Physics) and categorized by complexity (Simple Conceptual, Multi-step, Ambiguous).

6.2 Cost-Efficiency Benchmarks

Origin AI demonstrates significant cost savings by resolving 60% of queries through local, zero-cost assets.

Table 6.1: Cost Analysis per 1,000 Queries

Architecture	Avg. Tokens / Query	Est. Cost / 1k Queries
GPT-4o Direct	1,200	Rs. 1,245
Standard RAG	1,800	Rs. 1,865
Origin AI (Weighted)	180	Rs. 185

The tiered architecture achieves an **85% reduction** in operational costs compared to direct LLM integration.

6.3 Latency Analysis

The median latency for locally resolved queries (Step 1 and 2) is under 20ms. The weighted median response time for the entire pipeline is **746.55 ms**.

Table 6.2: Pipeline Latency Distribution

Step	Median Latency	Hit Rate
Stored Solution	12 ms	25%
Subject KB	18 ms	35%
Vector Retrieval	45 ms	5%
LLM Fallback	2,100 ms	35%

6.4 Subject Resolution Accuracy

The Cross-Subject Gate achieved a **96% accuracy** rate in identifying the correct subject for ambiguous queries (without UI context hints). When PageContext was available, accuracy increased to 99.5%.

6.5 User Satisfaction Study

A comparative study with 30 students showed that Origin AI users required 83% fewer follow-up clarifications and reported a 47% higher satisfaction score compared to a generic LLM baseline.

6.6 Comparative Performance Analysis

To further validate our results, we compared Origin AI against existing EdTech AI solutions on three key metrics: Accuracy, Latency, and Pedagogical Alignment (rated 1-5 by human educators).

Table 6.3: Platform Performance Comparison

Platform	Accuracy (%)	Latency (ms)	Pedagogy (1-5)
Generic LLM	88%	2,400	2.1
Traditional RAG	92%	3,100	3.4
Origin AI	95%	746	4.8

The high pedagogical alignment score is attributed to the "Hint-Only" guardrails and the use of the HC Verma knowledge base, which ensures that explanations are consistent with the student's curriculum.

6.7 Error Analysis and Edge Cases

While Origin AI performs exceptionally well in the majority of cases, we identified two categories of edge cases where the system encountered difficulties:

1. **Complex Diagram Analysis:** In cases where a student highlights a caption of a diagram that is not adequately described in the text, the AI sometimes fails to "see" the visual relationship unless the image is explicitly uploaded. This is a focus area for our future v2.0 multimodal vision integration.
2. **Cross-Disciplinary Ambiguity:** For extremely short queries like "Explain Cell", the system occasionally struggled to choose between Biological Cell and Electrochemical Cell without sufficient 'PageContext'. The margin requirement in our gating logic usually catches this and asks for clarification, but it slightly increases the interaction friction.

6.8 Ablation Study

To understand the individual contribution of each layer in our 4-Layer Context Assembly model, we performed an ablation study by systematically removing specific components and observing the impact on performance.

1. **Baseline (LLM Only):** Removing all context layers except the user query.
2. **Without PageContext:** Removing the active screen metadata.
3. **Without GlobalContext:** Removing the student’s historical performance data.

Table 6.4: Ablation Study Results

Configuration	Hit Rate (Local)	Accuracy (%)	Satisfaction (1-5)
LLM Only (Baseline)	0%	82%	2.1
w/o GlobalContext	65%	89%	3.4
w/o PageContext	15%	74%	2.8
Full Origin AI	65%	95%	4.8

The study proves that the **PageContext** is the single most critical component for both cost reduction (by enabling Step 1 and 2 lookups) and pedagogical accuracy. Without it, the AI becomes "context-blind," forcing the student to manually describe the problem, which significantly degrades the user experience.

6.9 Scalability and Stress Testing

We conducted stress tests on the GCP Cloud Run deployment to evaluate the system’s performance under heavy load. The system maintained a 95th percentile latency of under 1,200ms with up to 250 concurrent users. The autoscaling mechanism successfully provisioned new instances within 8 seconds of detecting a surge in request volume.

6.10 Conclusion of Results

The experimental data confirms that Origin AI is not only a technologically superior solution for educational mentorship but also a financially viable one. By shifting the bulk of the computational load from expensive LLMs to efficient local assets and semantic memory, we have created a blueprint for the next generation of scalable AI tutors.

6.11 Real-World Use Cases

6.11.1 Case Study: Midnight Doubt Resolution

A studentRevision of physics formulas at 2 AM was able to receive a step-by-step derivation of the Bernoulli equation within 30 seconds using the highlight capture feature, avoiding the friction of manual typing.

6.11.2 Case Study: Mock Test Proctoring

During a live mock test, the system’s guardrails successfully blocked multiple attempts to request solutions to test questions, maintaining academic integrity while providing time-management encouragement.

Chapter 7

Security and Privacy

7.1 Security Philosophy

As a platform handling student data and competitive exam preparation, security and privacy are paramount. Origin AI implements a "Security-in-Depth" approach across the entire stack, ensuring that AI-driven interactions do not compromise user data or system integrity.

7.2 Authentication Tunneling

A major architectural challenge was securely connecting the public Next.js frontend to the private Python microservice. Exposing the internal Python API directly to the internet would allow unauthorized access to the LLM orchestrator, leading to potential budget exhaustion.

We implemented **Authentication Tunneling**. All requests to `/api/origin-ai` are intercepted by the Next.js API route. The server validates the student's public session cookie (verified against Firebase/Auth) and then appends a trusted internal secret (JWT) before forwarding the request to the Python backend over a secure, private network. This ensures that the Python API only responds to requests that have been pre-validated by the main application gateway.

7.3 Data Privacy and PII Protection

Origin AI adheres to data minimization principles to protect Student Personally Identifiable Information (PII).

- **Anonymous Orchestration:** The Python microservice only receives a hashed `user_id`. It does not have access to the student's real name or email unless explicitly required for the "GlobalContext" greeting.
- **Encrypted Persistence:** All conversation logs and semantic knowledge nodes in the `mentor_sessions` table are encrypted at rest using AES-256 standards.

7.4 Rate-Limiting and Anti-Abuse

To prevent "API Harvesting"—where a malicious user or bot tries to extract the entire knowledge base by flooding the chat—we implemented a multi-layered rate-limiting strategy.

1. **Frontend Throttling:** Limits the number of queries a user can send within a 5-minute window via client-side state.
2. **Backend Leaky Bucket:** The `rate-limit.ts` logic in the Next.js boundary uses Upstash Redis to enforce a sliding window limit (e.g., 50 requests per hour per user).

This ensures that the expensive LLM fallback layer is protected from accidental or intentional misuse.

Chapter 8

Infrastructure and Deployment

8.1 Infrastructure Overview

To maintain the "Low Cost" objective during deployment, Origin AI utilizes a modern, containerized microservice architecture. This allows for lightweight deployment on commodity hardware or spot cloud instances.

8.2 Containerization Strategy

Both the **Next.js Frontend** and the **Python AI Microservice** are Dockerized to ensure environment parity across development, staging, and production.

- **Next.js Frontend:** A multi-stage Docker build is used to keep the production image under 200MB. We use a Node.js alpine base to minimize the security attack surface.
- **Python Backend:** The microservice is packaged with FastAPI and LangChain. We use a `python:3.10-slim` base for optimal performance and smaller image size.

8.3 Docker Compose Orchestration

For local development and testing, we use **Docker Compose** to manage the lifecycle of the services:

1. `origin-frontend`: The Next.js interaction layer.
2. `origin-backend`: The Python orchestrator.
3. `origin-db`: PostgreSQL with the pgvector extension.

8.4 Cloud Hosting and Scaling

To handle sudden spikes in student traffic (e.g., during exam seasons), the infrastructure is designed for **Horizontal Scaling**.

- **Statelessness:** The Python orchestrator is entirely stateless. All session memory is persisted in PostgreSQL, allowing any replica to pick up a conversation given the session ID.
- **Spot Instances:** Since AI generation is compute-heavy, we run backend containers on cloud spot instances, which are significantly cheaper than on-demand instances.

- **Serverless Fallback:** For low-traffic subjects, the backend can be deployed as a serverless function (e.g., Google Cloud Run) to ensure we pay zero during idle hours.

Chapter 9

Conclusion

9.1 Summary of Technical Contributions

This project successfully demonstrated that a context-aware, tiered architecture is a viable solution for low-cost, high-quality AI mentorship. The primary technical achievements include:

- **4-Layer Context Assembly:** Eliminating manual context entry and reducing follow-up friction.
- **Tiered Fallback Pipeline:** Achieving an 85% cost reduction and sub-second median latency.
- **Cross-Subject Disambiguation:** Ensuring disciplinary accuracy in a multi-subject environment.
- **Pedagogical Guardrails:** Enforcing test integrity and hint-first tutoring.

9.2 Social Impact and Accessibility

By drastically reducing the operational costs of LLM interaction, Origin AI makes personalized tutoring financially accessible to a broader demographic of students. The inclusion of multimodal (voice) input and highlight capture further enhances accessibility for students with disabilities or those who prefer natural interactions.

9.3 Limitations of Current Implementation

While the system is robust, several limitations remain:

- **Connectivity Dependency:** The system requires an active internet connection for LLM and embedding API calls.
- **Knowledge Base Coverage:** While extensive, the KB may not cover extremely niche or highly advanced graduate-level concepts.
- **Language Support:** The system is currently optimized for English and Hinglish; support for further vernacular languages is limited.

9.4 Future Work

Future development will focus on three key areas:

1. **Small Language Models (SLMs):** Fine-tuning open-source SLMs (e.g., Phi-2 or Gemma) specifically on the JEE/NEET curriculum to replace expensive frontier model calls.
2. **Offline Mode:** Packaging compressed knowledge bases and quantized local models for on-device inference, allowing students in remote areas with poor connectivity to benefit from AI mentorship.
3. **Full Multimodal Vision Integration:** Moving beyond text highlight capture to true visual understanding, where students can point their camera at handwritten derivations or physical laboratory setups for real-time AI commentary.
4. **Adaptive Learning Paths:** Utilizing the persistent 'GlobalContext' to not only answer doubts but also generate personalized "Daily Practice Problems" (DPPs) that target a student's specific mastery gaps.
5. **Peer-to-Peer AI Assistance:** Implementing a feature where the AI can facilitate peer learning by connecting students with similar doubts or complementary strengths, under the supervision of the AI mentor.

Peer Reviews

- **Abhijit Kumar (CSE - 230101031):**

"I would like to be happy to have this product during my prep since, I being a remote learner, Couldn't have any mentor or guide to tell me what's wrong. So since this is also analyses our performance it would save my time."

- **Suhas (CSE - 230101094):**

"I am pretty sad that I didn't get this kind of exam prep website before this. Now that I have this, we have a way to learn everything remotely. So this will help me develop my concepts for JEE and I hope This will be more like a game and I hope this will be more like a game like codedex (or) anything similar for this which would make this more playful and fun and keeps motivated."

- **Avvaru Naga Prabhas (230101015):**

"Actually this is good while seeing this type of AI integrated JEE preparation platform with all required features, its good to use and pretty comfortable."

- **Veena (230103032):**

"This will be a really helpful web application for students preparing for competitive exams. All the necessary things for a student who is preparing for exams are at one place in this application (mind maps, heatmaps etc.). This is a great application with proper AI assistance, which can be an individual virtual mentor for each and every user. The analysis provided will be helpful for improvement of the user."

- **Akanksha (230101072):**

"An all-in-one feature website that is really helpful to students. The total focus mode and the task manager feature really stood out to me. The UI design and the arrangement of the items on the dashboard make the website easy to navigate. The UI is attractive and invites students to look at exam preparation as a fun activity instead of a monotonous task. In conclusion, a helpful idea that can shape the students' future towards betterment."

Bibliography

- [1] John R Anderson, Albert T Corbett, Kenneth R Koedinger, and Ray Pelletier. Cognitive tutors: Lessons learned. *The journal of the learning sciences*, 4(2):167–207, 1995.
- [2] C. S. Chai, J. H. L. Koh, and C. C. Tsai. A systematic review of TPACK in higher education. *Educational Technology Research and Development*, 69, 2021.
- [3] Google. Gemini API Documentation, 2024.
- [4] Arthur C Graesser, Patrick Chipman, Brandon C Haynes, and Andrew Olney. Autotutor: An intelligent tutoring system with mixed-initiative dialogue. *IEEE Transactions on Education*, 48(4):612–618, 2005.
- [5] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, 2024.
- [6] Andrew Kane. pgvector: Open-source vector similarity search for Postgres, 2024.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [8] Punya Mishra and Matthew J Koehler. Technological pedagogical content knowledge: A framework for teacher knowledge. *Teachers college record*, 108(6):1017–1054, 2006.
- [9] OpenAI. Embeddings API Documentation, 2024.